

Docker Cheat Sheet

30 tables · 341 concepts · Updated 2026-09-27 · cheatgrid.com

Docker is an open platform for developing, shipping and running applications, packaging each app with what it needs into a container run from an image. The `docker` client sends commands over a REST API to the Docker daemon (`dockerd`), which builds, runs and distributes containers and manages images, networks and volumes; Docker Compose is another client for apps made of several containers. The same CLI now also carries Buildx, Scout, Model Runner (`docker model`), the MCP Toolkit (`docker mcp`), Offload and Hardened Images (`docker dhi`), and Docker Engine is on its 29.x line, which deprecated cgroup v1 support. The model that pays off most: an image is a **read-only template** built one layer per Dockerfile instruction, a container is a runnable instance of it, and whatever a container writes outside persistent storage disappears when the container is removed, so rebuilds only redo changed layers and data belongs in volumes.

CONTENTS

Container Lifecycle Management	2
Container Inspection and Interaction	3
Image Management	3
Dockerfile Instructions	4
Build Optimization and BuildKit	6
Registries, Authentication and Tagging	7
Port Publishing	8
Network Modes	8
Network Management	9
Volumes and Mounts	10
Resource Constraints	11
Restart Policies and Health Checks	12
Signal Handling	13
Logging Drivers	13
Inspect and Debug	14
Cleanup and Maintenance	15
Docker Compose Services	15
Docker Compose Commands	17
Docker Compose Advanced Features	18
Security Practices	19
Distroless and Minimal Images	20
Docker DHI CLI	21
Docker Scout Commands	22
Context and Remote Docker	22
Daemon and Engine Configuration	23
Docker Swarm Basics	24
Docker Build Cloud and Docker Offload	25
Docker Model Runner	26
Docker AI (Gordon)	27
Docker MCP Toolkit	27

References28

TABLE 1

Container Lifecycle Management

These commands move a container through every phase of its life, from initial creation, through running and pausing, to graceful or forced termination. Knowing which command uses which signal (and which preserves the writable layer) is what separates clean production shutdowns from corrupted state.

Command	Example	Description
docker run	<code>docker run -d -p 8080:80 --name web nginx</code>	- Creates and starts a new container from an image <code>-d</code> runs detached, <code>-p</code> maps ports, <code>--name</code> assigns a custom name. Most commonly used command.
--rm flag	<code>docker run --rm -it ubuntu bash</code>	Automatically removes the container and its anonymous volumes when it exits - ideal for short-lived, throwaway containers.
-it flags	<code>docker run -it debian bash</code>	<code>-i</code> keeps STDIN open, <code>-t</code> allocates a pseudo-TTY - combined they give an interactive terminal session inside the container.
-e flag	<code>docker run -e NODE_ENV=prod nginx</code>	- Sets single runtime variable at container start - visible in <code>docker inspect</code>, so unsuitable for secrets.
--env-file	<code>docker run --env-file .env nginx</code>	- Loads variables from a file, one <code>VAR=value</code> per line <code>#</code> starts a line comment; no shell-style <code>\$VAR</code> interpolation.
docker start	<code>docker start web</code>	- Starts an existing stopped container - preserves all container state and configuration.
docker stop	<code>docker stop web</code>	Gracefully stops a running container by sending SIGTERM then SIGKILL after grace period (default 10s).
--timeout (-t)	<code>docker stop -t 30 web</code>	- Sets the seconds to wait before the SIGKILL on <code>stop</code> or <code>restart</code> - replaces the deprecated <code>--time</code> flag; <code>-1</code> waits indefinitely.
docker restart	<code>docker restart web</code>	Stops then starts the same container instance - equivalent to <code>stop + start</code>; preserves the writable layer.
docker pause	<code>docker pause web</code>	Freezes all processes in a container using the cgroups freezer - no CPU usage, but memory stays allocated; instant resume with <code>unpause</code>.
docker unpause	<code>docker unpause web</code>	Resumes a paused container - restores execution exactly where it stopped.
docker rm	<code>docker rm -f web</code>	Removes a stopped container <code>-f</code> forces removal of running containers by SIGKILL-ing them first (no grace period).
docker kill	<code>docker kill web</code>	- Immediately stops a container by sending SIGKILL - no graceful shutdown, use for unresponsive containers.
docker create	<code>docker create --name web nginx</code>	- Creates a container without starting it - useful for preparing containers before <code>start</code>.
docker wait	<code>docker wait web</code>	- Blocks until a container stops, then prints its exit code - useful in scripts.
docker rename	<code>docker rename web web-old</code>	Renames an existing container - works on running and stopped containers; log driver tags keep the original name.
docker update	<code>docker update --cpus 2 --memory 1g web</code>	- Dynamically updates resource limits and restart policy on running containers - cannot change image, env vars, ports, or volumes.

TABLE 2

Container Inspection and Interaction

Once a container is running, day-to-day work is about seeing what it is doing and reaching inside it. These commands cover listing containers, reading their logs and metrics, running ad-hoc commands inside them, and pulling out the metadata Docker keeps about each one.

Command	Example	Description
docker ps	<code>docker ps -a</code>	- Lists containers - default shows running only, <code>-a</code> includes stopped; <code>--filter health=unhealthy</code> filters by health state; <code>--format '{{.HealthStatus}}'</code> prints health as a dedicated field (Engine 29.5+).
docker logs	<code>docker logs -f --tail 100 web</code>	- Shows container stdout/stderr of the main process <code>-f</code> follows live output, <code>--tail</code> limits lines <code>--since</code> / <code>--until</code> narrow output to a time window, <code>--timestamps</code> adds an RFC3339Nano stamp to each line.
docker exec	<code>docker exec -it web bash</code>	- Runs a new process inside a running container <code>-it</code> allocates interactive TTY for shell access.
docker attach	<code>docker attach web</code>	- Connects to container's main process (PID 1) stdin/stdout/stderr - Ctrl+C sends SIGINT and usually stops the container; detach with Ctrl+P, Ctrl+Q.
docker inspect	<code>docker inspect --format='{{.State.Health.Status}}' web</code>	- Returns detailed JSON with a container's own state, health and exit status, mounts and IPs <code>--format</code> Go templates pull out one field at a time.
docker stats	<code>docker stats --no-stream web</code>	- Live stream of CPU, memory, network, disk I/O metrics <code>--no-stream</code> prints one snapshot then exits (use in scripts).
docker top	<code>docker top web</code>	- Shows running processes inside a container, using the host's <code>ps</code> - PID column reports host PIDs, not the container's PID 1.
docker port	<code>docker port web</code>	- Lists all published port mappings for a container - shows host port to container port bindings (only ports published with <code>-p</code> / <code>-P</code>).
docker diff	<code>docker diff web</code>	Shows filesystem changes vs the starting image (A=added, C=changed, D=deleted).
docker cp (container to host)	<code>docker cp web:/app/log.txt ./log.txt</code>	- Copies files from container to host, works on running and stopped containers - directories are copied recursively by default (no <code>-r</code> flag needed).
docker cp (host to container)	<code>docker cp ./config.yml web:/app/config.yml</code>	- Copies file from host to container - immediately available in running container.
docker cp (stream via stdin/stdout)	<code>docker cp web:/var/logs/app.log - tar x -0 grep "ERROR"</code>	<code>-</code> as <code>SRC_PATH</code> or <code>DEST_PATH</code> streams the copy as a tar archive over stdin/stdout - lets <code>docker cp</code> output feed straight into another command.
Archive mode	<code>docker cp -a web:/app ./backup</code>	- Preserves UID/GID and permissions - equivalent to <code>cp -a</code>.

TABLE 3

Image Management

Commands for getting images onto and off your machine: building from a Dockerfile, pulling and pushing through registries, tagging, inspecting layers, and moving images between hosts as tar archives. The two biggest gotchas live here too, `save` vs `export` (image vs container, layered vs flat) and `docker commit` as a production anti-pattern.

Command	Example	Description
docker build	<code>docker build -t myapp:1.0 .</code>	- Builds an image from a Dockerfile <code>-t</code> tags it; <code>.</code> is the build context directory sent to the daemon.

Command	Example	Description
docker images	<code>docker images</code> <code>docker images --filter dangling=true</code>	- Lists local images with repository, tag, image ID, creation date, and size <code>--filter</code> narrows the list by <code>dangling</code>, <code>label</code>, <code>before</code> / <code>since</code>, or a <code>reference</code> pattern.
docker pull	<code>docker pull nginx:alpine</code> <code>docker pull nginx@sha256:2e863c...</code>	- Downloads an image from a registry (default Docker Hub) - only missing layers are fetched thanks to content-addressable storage - pulling by digest pins an exact, immutable version instead of a movable tag.
docker push	<code>docker push myuser/myapp:1.0</code>	- Uploads an image to a registry - requires <code>docker login</code> first.
docker tag	<code>docker tag myapp:1.0 myapp:latest</code>	- Adds a new name pointing to the same image ID - does not copy or duplicate layers.
docker rmi	<code>docker rmi nginx:alpine</code>	Removes an image - fails if any container (running or stopped) references it unless <code>-f</code> forces removal.
docker history	<code>docker history nginx</code>	- Shows each layer in an image with its size and creating command - the go-to tool for diagnosing image bloat.
docker save	<code>docker save -o app.tar myapp:1.0</code>	Exports an image to a tar archive, preserving all layers and metadata.
docker load	<code>docker load -i app.tar</code>	- Imports an image from a <code>docker save</code> tarball - restores all layers and original tags.
docker import	<code>docker import rootfs.tar myapp:1.0</code>	- Creates an image from a raw filesystem tarball - result is a single layer with no build history.
docker export	<code>docker export web -o web.tar</code>	- Exports a container's filesystem as a flat tar - layers collapsed, image metadata lost.
docker commit	<code>docker commit web myapp:snapshot</code>	- Creates a new image from a container's filesystem changes - fine for ad-hoc debugging, an anti-pattern for production (no Dockerfile, not reproducible) - pauses the container by default while committing; <code>--no-pause</code> skips that.
docker buildx imagetools inspect	<code>docker buildx imagetools inspect alpine</code>	- Shows a registry image's manifest list, the index of per-architecture manifests behind a multi-platform image <code>--raw</code> prints the unformatted JSON manifest.
docker search	<code>docker search --limit 5 nginx</code>	- Searches Docker Hub only (not arbitrary registries) <code>--filter</code> narrows by stars or official status.

TABLE 4

Dockerfile Instructions

A Dockerfile is a recipe of instructions that BuildKit executes top-to-bottom to assemble an image, with most instructions producing a new layer. The instructions below are the building blocks every Dockerfile uses, and the trickiest part is knowing which ones run at build time versus runtime, and which ones only attach metadata.

Instruction	Example	Description
Parser directives	<code># syntax=docker/dockerfile:1</code> <code># escape=\\</code>	- Special comments on the first lines of a Dockerfile, before any other comment or instruction <code>syntax</code> pins the Dockerfile frontend version; <code>escape</code> changes the line-continuation character from a backslash to a backtick (useful on Windows)
FROM	<code>FROM node:18-alpine</code>	- Sets the base image and starts a build stage - every Dockerfile begins with FROM (after optional ARG/parser directives) - pin a specific tag or digest, never <code>latest</code> in production.

Instruction	Example	Description
RUN	<code>RUN <<EOF apt-get update apt-get install -y curl EOF</code>	- Executes commands during image build and commits the result as a new layer - chain related commands with <code>&&</code> or a heredoc in the same RUN to avoid stale-cache bugs - supports heredocs to break up long instructions without backslash line continuations
CMD	<code>CMD ["npm", "start"]</code>	- Provides the default command (or default args for ENTRYPOINT) when a container starts - replaced by any command passed to <code>docker run</code> - prefer the exec form (JSON array) for correct signal handling.
ENTRYPOINT	<code>ENTRYPOINT ["python", "app.py"]</code>	- Configures the container to run as a fixed executable - args to <code>docker run</code> are appended to it; override only with <code>--entrypoint</code> - when both ENTRYPOINT and CMD are exec form, CMD becomes default arguments.
COPY	<code>COPY package*.json ./</code>	- Copies files and directories from the build context into the image - preferred default for local files: predictable, no hidden behavior - supports heredocs to write inline file content (<code>COPY <<EOF file.txt</code>)
ADD	<code>ADD app.tar.gz /app/</code>	- Like COPY, but also auto-extracts local tar archives and can fetch remote URLs - use COPY unless you specifically need extraction or URL/Git fetching.
WORKDIR	<code>WORKDIR /app</code>	- Sets the working directory for the RUN, CMD, ENTRYPOINT, COPY, and ADD that follow - creates the directory if missing, prefer absolute paths.
ENV	<code>ENV NODE_ENV=production</code>	- Sets environment variables that persist in subsequent build steps and in running containers - never use for secrets: values leak into image layers and <code>docker history</code> .
ARG in Dockerfile	<code>ARG VERSION=1.0</code>	Build-time only, passed with <code>--build-arg</code> - not in the final image's runtime env, but still visible in <code>docker history</code> .
EXPOSE	<code>EXPOSE 8080</code>	Documents which ports the container listens on (image metadata only) - does NOT publish ports: <code>docker run -p</code> or <code>-P</code> is still required for host access.
VOLUME	<code>VOLUME /data</code>	- Declares a path as a mount point for an external/anonymous volume - with the legacy builder, writes to that path in later RUN steps are discarded (kept under BuildKit).
USER	<code>USER node</code>	- Sets the UID/GID used by subsequent RUN, CMD, and ENTRYPOINT and at runtime - place it after any RUN that needs root (package install, chown), since running as non-root is a security baseline.
LABEL	<code>LABEL version="1.0"</code>	- Attaches metadata to the image as key=value pairs - visible via <code>docker inspect</code> and filterable with <code>docker image ls --filter</code> , no runtime behavior.
HEALTHCHECK	<code>HEALTHCHECK CMD curl -f http://localhost/ exit 1</code>	- Defines a command Docker runs periodically to test if the container is healthy (exit 0=healthy, 1=unhealthy, 2 reserved) - only flips the container's health status; Compose/Swarm/K8s decide whether to restart.
STOPSIGNAL	<code>STOPSIGNAL SIGQUIT</code>	- Sets the signal sent to PID 1 when stopping the container - defaults to SIGTERM; signals only reach the process directly when CMD/ENTRYPOINT use exec form.
SHELL	<code>SHELL ["/bin/bash", "-c"]</code>	- Overrides the shell used for shell-form RUN/CMD/ENTRYPOINT - default is <code>["/bin/sh", "-c"]</code> on Linux and <code>["cmd", "/S", "/C"]</code> on Windows, useful for switching to PowerShell.
ONBUILD	<code>ONBUILD COPY . /app</code>	- Registers a trigger instruction that runs only when this image is used as a FROM base in a downstream build - useful for framework base images; triggers do not cascade to grandchildren.

TABLE 5

Build Optimization and BuildKit

These techniques shrink final images, speed up rebuilds, and keep secrets out of layer history. Most are built on BuildKit, the modern Dockerfile frontend (`# syntax=docker/dockerfile:1`): multi-stage builds, cache mounts, and secret mounts only behave as described when BuildKit is the active builder.

Technique	Example	Description
Multi-stage builds	<pre>FROM golang AS builder FROM alpine COPY --from=builder /app .</pre>	- Uses multiple FROM statements to create separate build stages; only artifacts copied into the final stage end up in the published image - intermediate stages (compilers, dev tools) are discarded entirely, drastically reducing size and attack surface.
Build targets	<pre>docker buildx build --target build-env -t myapp .</pre>	<code>--target</code> builds only up to a named intermediate stage, skipping every stage after it - lets a multi-stage Dockerfile double as a <code>debug</code> or <code>test</code> image without producing the final stage.
Layer caching	<pre>COPY package*.json ./ RUN npm install COPY . .</pre>	- Docker caches each instruction's output as a layer; if a layer's inputs change, that layer and every layer after it must rebuild - order instructions least- to most-frequently changed (manifests + install before <code>COPY . .</code>) to maximize hits.
COPY --link	<pre>COPY --link requirements.txt .</pre>	- Creates an independent layer that doesn't depend on earlier ones, so changes to instructions above it don't invalidate it - enables remote cache reuse (<code>--cache-from</code>) and image rebasing across multi-stage and large builds.
.dockerignore	<pre>node_modules *.log .git</pre>	- Excludes files from the build context (the bundle sent to the BuildKit daemon), not from the image directly - shrinks context upload, blocks accidental <code>COPY</code> of <code>node_modules</code> / <code>.git</code> / secrets, and keeps the cache stable across machines.
BuildKit cache mounts	<pre>RUN --mount=type=cache,target =/root/.npm npm install</pre>	- Mounts a persistent build-time cache at the target path for one <code>RUN</code> only; contents survive between builds but are never stored in image layers - target the package manager's download dir (<code>/root/.npm</code> , <code>/var/cache/apt</code> , <code>/root/.cache/pip</code>), not the install dir.
Build secrets	<pre>RUN --mount=type=secret,id=token curl -H "Auth: \$(cat /run/secrets/token)"</pre>	- Mounts a secret as a tmpfs file (e.g. <code>/run/secrets/token</code>) for a single <code>RUN</code> ; never written to layers, docker history , or build cache - replaces the leaky <code>--build-arg / ENV SECRET=...</code> pattern.
SSH mounts	<pre>RUN --mount=type=ssh git clone git@github.com:repo (involve with docker build --ssh default)</pre>	- Forwards the host's SSH agent socket into a single <code>RUN</code> step - lets builds clone private Git repos without a key entering the image.
RUN --mount=type=bind	<pre>RUN --mount=type=bind,target=. go build -o /app/hello</pre>	- Mounts build-context files into the step without COPY - files are read-only by default and never enter image layers or the cache.
Combining RUN commands	<pre>RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*</pre>	- Chain commands with <code>&&</code> so the install and the cleanup happen in the same layer, since files deleted in a later <code>RUN</code> still live in earlier layers' history - also prevents the stale- <code>apt-get update</code> caching trap.
Buildx for multi-platform	<pre>docker buildx build --platform linux/amd64,linux/arm64 -t app .</pre>	<code>buildx</code> (the BuildKit-backed CLI) builds for multiple architectures in one invocation, producing a single multi-arch manifest - cross-arch builds need either QEMU emulation or native builder nodes per platform.

Technique	Example	Description
External cache	<code>docker buildx build --cache-from type=registry,ref=user/app:buildcache --cache-to type=registry,ref=user/app:buildcache,mode=max .</code>	- Pushes/pulls build cache to a remote location (registry, S3, GHA) - lets ephemeral CI runners share cache and stay fast.
Inline cache	<code>docker buildx build --cache-to type=inline --cache-from <image> -t app .</code>	- Embeds cache metadata inside the published image itself - simplest registry-backed cache, but only supports <code>mode=min</code> .
Build checks	<code>docker buildx build --check .</code>	<code>--check</code> (shorthand for <code>--call=check</code>) validates the Dockerfile against build rules without running the build - catches base-image and syntax mistakes before a real build starts.
SBOM attestation	<code>docker buildx build --sbom=true -t myapp .</code>	- Attaches an SPDX Software Bill of Materials as an OCI attestation - lists packages in the image so scanners (Scout, Trivy) can flag CVEs.
Provenance attestation	<code>docker buildx build --provenance=mode=max -t myapp .</code>	- Attaches a SLSA-format provenance record proving what source, frontend, and parameters produced the image <code>mode=max</code> adds the Dockerfile and LLB; <code>mode=min</code> is the default.
Buildx bake	<code>docker buildx bake -f docker-bake.hcl</code> (HCL defines <code>target</code> and <code>group</code> blocks)	- Runs multiple build targets concurrently from an HCL/JSON/Compose file - supports variables, inheritance, and matrix targets for monorepos and multi-arch fan-out.

TABLE 6

Registries, Authentication and Tagging

Authenticating to a Docker registry sets up the credentials that later `push` and `pull` commands silently reuse, so the way you log in directly shapes how exposed those secrets are. This table covers the day-to-day commands and the two safer non-interactive patterns to prefer over `-p` .

Command	Example	Description
docker login	<code>docker login registry.example.com</code>	Authenticates to a registry and caches credentials for later <code>push</code> / <code>pull</code> - stores them via the configured credential helper, or base64-encoded in <code>~/.docker/config.json</code> as a fallback.
docker logout	<code>docker logout registry.example.com</code>	- Removes the locally stored credential for the given server (or the default registry) - does not revoke the token on the registry: rotate it there if it may have leaked.
Credential helpers (credsStore)	<code>{ "credsStore": "osxkeychain" }</code>	- Points <code>~/.docker/config.json</code> at an OS keychain helper (<code>osxkeychain</code> , <code>wincred</code> , <code>pass</code>) instead of storing credentials base64-encoded <code>credHelpers</code> sets a different helper per registry.
Username/password login	<code>docker login -u user -p pass</code>	- Non-interactive login with the password as a plain CLI argument avoid: it lands in shell history and log files, unlike the STDIN forms below.
Token authentication	<code>echo \$TOKEN docker login -u user --password-stdin</code>	- Reads the password or PAT from stdin so it never appears in history or process listings - the recommended pattern for CI and short-lived cloud tokens (ECR, GCR, ACR) - Engine 29 also accepts <code>--password -</code> as an alternative.
Semantic versioning	<code>myapp:1.2.3</code>	- Uses major.minor.patch format - best practice for release tracking and rollbacks.
Git commit SHA	<code>myapp:sha-a3f2c1b</code>	- Tags with the short Git commit hash - enables exact source code traceability, commonly generated by CI tagging actions.
Branch tags	<code>myapp:feature-auth</code>	- Tags with the Git branch name (slashes become dashes) - tracks feature development images per branch.

Command	Example	Description
Date tags (nightly)	<code>myapp:20260301</code>	- Timestamps with a YYYYMMDD pattern on a schedule trigger - useful for nightly builds or time-based releases.
Image digests (immutable)	<code>docker pull nginx@sha256:94a0...ea1a</code>	- A SHA-256 hash of the image content, unlike a tag it never changes once built - pin deployments to a digest when you need the exact same image every time.

TABLE 7

Port Publishing

Container ports stay isolated from the host until you explicitly publish them. The `-p` family of flags wires a host address and port to a container port; getting the syntax, especially the host-then-container order, the default `0.0.0.0` bind, and the implicit TCP protocol, right is the difference between a reachable service and a silent failure.

Option	Example	Description
-p (publish)	<code>docker run -p 8080:80 nginx</code>	- Maps a host port to a container port using <code>HOST_PORT:CONTAINER_PORT</code> - with no host IP, binds to <code>0.0.0.0</code> on every interface.
-P (publish-all)	<code>docker run -P nginx</code>	- Publishes every <code>EXPOSE</code> d port to a random ephemeral host port - query the chosen ports with <code>docker port <container></code> .
Specific host IP	<code>docker run -p 127.0.0.1:8080:80 nginx</code>	- Binds the published port to a specific host interface - use <code>127.0.0.1</code> to restrict access to the host (e.g. behind a reverse proxy).
Docker and UFW	<code>docker run -p 8080:80 nginx</code>	- Docker inserts its own firewall rules (iptables or nftables) ahead of <code>ufw</code> 's chains - a published port stays reachable even if a <code>ufw deny 8080</code> rule is active.
UDP ports	<code>docker run -p 53:53/udp dns</code>	<code>-p</code> publishes TCP by default; append <code>/udp</code> for UDP services like DNS, QUIC, or SIP - to publish both, repeat the flag: <code>-p 53:53/tcp -p 53:53/udp</code> .
Port range	<code>docker run -p 8000-9000:5000 app</code>	- Maps a range of host ports to a container port, or a range-to-range one-to-one <code>8000-9000:5000</code> binds container port 5000 to the first free host port in that range.
IPv6 published ports	<code>docker run -p '[::1]:8080:80' -p 127.0.0.1:8080:80 nginx</code>	<code>-p</code> publishes on both IPv4 and IPv6 by default <code>[::1]:port:port</code> binds only the IPv6 loopback, matching <code>127.0.0.1</code> for IPv4.

TABLE 8

Network Modes

Docker networking is driven by a pluggable set of built-in drivers: each one shapes how a container's network namespace is wired up, what it can reach, and how it appears to the rest of the world. Picking the right one is the difference between a quick local test and a setup that survives a multi-host production deployment.

Mode	Example	Description
bridge (default)	<code>docker run --network bridge nginx</code>	- Containers get private IPs on a virtual L2 bridge with NAT for outbound traffic - the default bridge has no name-based DNS between containers: only user-defined bridge networks do.
host	<code>docker run --network host nginx</code>	- Container shares the host's network namespace: no isolation, no NAT, binds host ports directly - fastest but largest security blast radius. Linux only (Desktop 4.34+ via VM bridge).
none	<code>docker run --network none app</code>	- Isolated network namespace with only lo (loopback) - no other interfaces, no external connectivity, useful for batch jobs that must not touch the network.
Container network mode	<code>docker run --network container:redis busybox</code>	- Attaches to another container's network namespace, sharing its IP and ports - incompatible with <code>-p</code> , <code>--hostname</code> , <code>--dns</code> and other network-scoped flags on the joining container.

Mode	Example	Description
overlay	<code>docker network create -d overlay --attachable my-net</code>	- Multi-host networking over VXLAN for Docker Swarm - requires <code>docker swarm init</code> even for single-node use; containers on different swarm nodes see each other as if on one LAN.
macvlan	<code>docker network create -d macvlan --subnet=192.168.1.0/24 -o parent=eth0 my-net</code>	- Each container gets its own MAC and appears as a physical device on the parent NIC's LAN - Linux-only; most cloud providers block it (hypervisor MAC-spoof protection).
ipvlan	<code>docker network create -d ipvlan --subnet=192.168.1.0/24 -o parent=eth0 my-net</code>	- Like macvlan but containers share the parent's MAC (L2 mode) - works where macvlan is blocked, and avoids upstream-switch MAC-table exhaustion at scale.
nftables firewall backend	<code>dockerd --firewall-backend=nftables</code>	Experimental (Engine 29.0+) alternative to the default iptables backend - not usable with Swarm mode; overlay-network rules have not yet been migrated.

TABLE 9

Network Management

Docker networking glues containers to each other and to the outside world. The commands here create and manage **user-defined networks** (almost always what you want, since they enable automatic container-name DNS, unlike the default `bridge`), connect and disconnect running containers, and reach back to the host machine from inside a container.

Command	Example	Description
docker network create	<code>docker network create mynet</code>	- Creates a user-defined network: default driver is <code>bridge</code> , use <code>-d overlay</code> for Swarm or <code>-d macvlan</code> for L2 - user-defined bridges enable automatic DNS resolution between containers by name (the default <code>bridge</code> network does not).
docker network ls	<code>docker network ls</code>	- Lists all networks with <code>NETWORK ID</code> , <code>NAME</code> , <code>DRIVER</code> , and <code>SCOPE</code> - always includes the three predefined networks <code>bridge</code> , <code>host</code> , and <code>none</code> that ship with the daemon.
docker network connect	<code>docker network connect mynet web</code>	- Attaches a running container to an additional network: containers can be on multiple networks at once - on the primary network they resolve each other by name; on secondary networks they must use IP.
Network aliases (--alias)	<code>docker network connect --alias db --alias mysql mynet web</code>	- Gives a container one or more extra names that other containers on the same network can resolve it by - resolved through Docker's embedded DNS server (<code>127.0.0.11</code>) alongside the container's own name.
docker network disconnect	<code>docker network disconnect mynet web</code>	- Removes a container from a network while it is still running: no need to stop it first - commonly used before <code>docker network rm</code> to free a network that's blocking removal.
docker network inspect	<code>docker network inspect mynet</code>	- Returns the full JSON config for a network: driver, IPAM Subnet / Gateway , every connected container, IP and MAC addresses, driver options - the first stop when containers can see each other by IP but not by name.
docker network rm	<code>docker network rm mynet</code>	- Removes a user-defined network: fails with "has active endpoints" if any container is still attached - disconnect or stop+remove the container first, then retry. <code>-f</code> only ignores "no such network", it does not force-detach.
host.docker.internal	<code>curl http://host.docker.internal:8000</code>	- Special DNS name that resolves to the internal IP of the host from inside a container: built in on Docker Desktop - on Docker Engine without Desktop it does not exist by default; add <code>--add-host host.docker.internal=host-gateway</code> to <code>docker run</code> .

Command	Example	Description
Legacy --link (legacy)	<code>docker network connect --link container1:c1 mynet container2</code>	- Links a container to another by a preferred alias on the default bridge network, replaced by user-defined networks and <code>--alias</code> - since Docker v29.0 the legacy env vars it injects are no longer set by default and are removed in v30.0.

TABLE 10

Volumes and Mounts

Containers are ephemeral, but most real applications need data that outlives any single container. Docker offers three storage primitives, named volumes, bind mounts, and tmpfs, each with different durability, portability, and host-coupling trade-offs, plus two CLI shapes (`-v` and `--mount`) whose subtle differences trip people up.

Type	Example	Description
Named volumes	<code>docker run -v data:/app/data nginx</code>	- Docker-managed storage under <code>/var/lib/docker/volumes/</code>, lifecycle independent of the container preferred for production; easy to back up, share between containers, and reference by name.
Bind mounts	<code>docker run -v /host/path:/container/path nginx</code>	- Maps an absolute host path straight into the container - changes sync in real time, ideal for development; portability and permission ownership are your problem, not Docker's.
tmpfs mounts	<code>docker run --tmpfs /tmp nginx</code>	- Stores data only in host RAM, never written to disk; removed when the container stops - good for caches and short-lived secrets, not for anything that must survive a restart.
--mount syntax	<code>docker run --mount type=volume,source=mydata,target=/data nginx</code>	Explicit, key-value alternative to <code>-v</code> that supports <code>type=volume</code>, <code>type=bind</code>, and <code>type=tmpfs</code> - recommended for scripting because it errors on a missing bind source instead of silently creating a directory.
Anonymous volumes	<code>docker run -v /app/data nginx</code>	- Created automatically with a random hash name when no source is given - persist after the container exits unless you used <code>--rm</code>; accumulate quickly and are hard to identify.
Read-only volumes	<code>docker run -v data:/app:ro nginx</code>	<code>:ro</code> (or <code>readonly</code> with <code>--mount</code>) makes the mount read-only inside the container - the host can still modify the underlying files; it is a container-side guard, not a host-side lock.
Back up a volume to a tar archive	<code>docker run --rm --volumes-from dbstore -v \$(pwd):/backup ubuntu tar cvf /backup/backup.tar /dbdata</code>	- Runs a throwaway container that mounts the target volume and tars its contents onto a bind-mounted host directory - the same pattern, reversed, restores a volume from that archive.
volumes-from	<code>docker run --volumes-from web nginx</code>	- Mounts all volumes from another container at their original paths - classic backup/restore trick; modern setups usually share a single named volume by name instead.
Image mounts	<code>docker run --mount type=image,source=busybox:musl,destination=/dbg alpine</code>	- Mounts a built image's filesystem read-only into a container with <code>--mount type=image</code> - needs the containerd image store; no <code>-v</code> equivalent, and the image must already be pulled locally.
Volume drivers (NFS, CIFS)	<code>docker volume create --driver local --opt type=cifs --opt device=//server/share --name cifs-volume</code>	- Plugs in a driver (built-in NFS/CIFS, or a third-party plugin) so a volume points at a remote share instead of local disk - lets containers on different hosts see the same data without a host-side mount point.
Bind propagation	<code>docker run -v ./project:ro,rshared</code>	- Controls whether mounts created inside a bind mount (<code>rprivate</code> default, <code>shared</code>, <code>slave</code>, and recursive variants) propagate to replica mounts bind mounts only, Linux hosts only; does not work on Docker Desktop.

Type	Example	Description
SELinux relabeling (:z/:Z)	<code>docker run -d -it --name devtest -v "\$(pwd)"/target:/app:z nginx:latest</code>	• Adds <code>:z</code> (shared) or <code>:Z</code> (private) to a bind mount to fix SELinux access denials on the host directory • relabels files on the host itself; only works with <code>-v</code>, not with <code>--mount</code>.
docker volume create	<code>docker volume create mydata</code>	• Creates a named volume with a chosen driver and options • rarely required for plain use, since <code>docker run -v mydata:/path</code> auto-creates it.
docker volume ls	<code>docker volume ls</code>	• Lists all volumes with driver and name • does not show which container uses each; pair it with <code>docker ps --filter volume=<name></code>.
docker volume inspect	<code>docker volume inspect mydata</code>	• Shows volume metadata including the Mountpoint • on Docker Desktop, that path lives inside the utility VM, not on the host filesystem.
docker volume rm	<code>docker volume rm mydata</code>	• Removes a volume • fails if any container references it, including stopped ones (check with <code>docker ps -a</code>).

TABLE 11

Resource Constraints

By default a Docker container can use as much memory and CPU as the host kernel will give it, which makes "noisy neighbor" problems and OOM cascades easy to trigger. These flags map directly onto Linux cgroup controllers (memory, CPU CFS, cpuset, pids, blkio) and let you pick between **hard caps**, **soft reservations**, **relative weights**, and **device pinning**: distinctions that decide whether a container crashes with exit 137 or simply slows down.

Flag	Example	Description
--memory (-m)	<code>docker run -m 512m nginx</code>	• Hard memory limit (cgroup) • OOM killer sends SIGKILL on breach (exit 137), suffixes: <code>b k m g</code>.
--memory-reservation	<code>docker run --memory-reservation 256m nginx</code>	• Soft limit only enforced under host memory pressure • must be lower than <code>--memory</code> to take effect; container may exceed it.
--memory-swap	<code>docker run -m 300m --memory-swap 1g nginx</code>	• Total memory plus swap the container may use; only meaningful together with <code>--memory</code> • set equal to <code>--memory</code> to disable swap, or <code>-1</code> for unlimited swap.
OOMKilled flag	<code>docker inspect --format '{{.State.OOMKilled}}' <container> dmesg grep -i "killed process"</code>	• true in <code>docker inspect</code> means the container's own cgroup limit triggered the kill (exit 137) • false with exit 137 plus an OOM line in <code>dmesg</code> / <code>journalctl -k</code> points to a host-wide memory shortage instead.
--oom-kill-disable	<code>docker run -m 512m --oom-kill-disable nginx</code>	• Stops the kernel from killing the container on OOM • only safe when <code>-m</code> / <code>--memory</code> is also set, otherwise the host can run out of memory instead.
--oom-score-adj	<code>docker run --oom-score-adj -500 nginx</code>	Tunes the container's OOM-kill priority relative to other host processes, from <code>-1000</code> (least likely) to <code>1000</code> (most likely).
--cpus	<code>docker run --cpus 1.5 nginx</code>	• Hard cap on total CPU time via CFS quota/period • <code>1.5 = --cpu-period=100000 + --cpu-quota=150000</code>; kernel picks which cores.
--cpu-shares	<code>docker run --cpu-shares 512 nginx</code>	• Relative weight (default 1024) for CFS, not a cap • only honored under CPU contention; idle host = no effect.
--cpuset-cpus	<code>docker run --cpuset-cpus 0,1 nginx</code>	• Pins the container to specific physical CPUs (cores 0 and 1) • used for NUMA locality and cache-sensitive workloads.

Flag	Example	Description
<code>--pids-limit</code>	<code>docker run --pids-limit 100 nginx</code>	• Caps total PIDs (processes + threads) via the pids cgroup • mitigates fork bombs; thread-heavy JVM/Python apps can hit it.
<code>--blkio-weight</code>	<code>docker run --blkio-weight 500 nginx</code>	• Relative block-I/O weight (default 500, range 10-1000) • effective only on schedulers that honor it (e.g., BFQ/CFQ) and under disk contention.
<code>--gpus</code>	<code>docker run -it --rm --gpus all ubuntu nvidia-smi</code>	• Exposes host GPUs to the container • requires the NVIDIA Container Toolkit on the host; use <code>--gpus device=0</code> or <code>--gpus '"device=0,2"'</code> to target specific GPUs.

TABLE 12

Restart Policies and Health Checks

Restart policies tell the Docker daemon when to bring a stopped container back on its own, after a crash, a manual stop, or a daemon restart; the `--health-*` flags then decide whether Docker calls that running container healthy in the first place. A policy only takes effect once a container has stayed up at least 10 seconds, and it is separate from the dockerd `--live-restore` flag, which keeps containers running through a Docker upgrade instead.

Policy	Example	Description
no (default)	<code>docker run --restart no nginx</code>	• Never restart container automatically • manual <code>docker start</code> still works.
on-failure	<code>docker run --restart on-failure:3 nginx</code>	• Restarts only on non-zero exit code • optional <code>:N</code> cap; not triggered by daemon restart.
always	<code>docker run --restart always nginx</code>	• Always restarts, including after daemon restart • brings back even manually-stopped containers on reboot.
unless-stopped	<code>docker run --restart unless-stopped nginx</code>	• Like <code>always</code> but respects manual stops across daemon restarts • preferred for single-host production stacks.
<code>--health-cmd</code>	<code>docker run --health-cmd "curl -f http://localhost/" nginx</code>	• Runtime override that attaches (or replaces) the probe command without rebuilding the image • useful for ad-hoc testing or images shipped without a <code>HEALTHCHECK</code>.
<code>--health-interval</code>	<code>docker run --health-interval=10s nginx</code>	• Wait time between checks (after the previous one completes), default <code>30s</code> • the first check also runs one interval after container start.
<code>--health-timeout</code>	<code>docker run --health-timeout=5s nginx</code>	• Max time for a single check to complete, default <code>30s</code> • if exceeded, the probe is <code>SIGKILL</code>-ed and that run counts as a failure.
<code>--health-retries</code>	<code>docker run --health-retries=3 nginx</code>	• Number of consecutive failures needed to flip the state to <code>unhealthy</code>, default <code>3</code> • one successful check resets the failing streak to zero.
<code>--health-start-period</code>	<code>docker run --health-start-period=60s nginx</code>	• Initialization grace window, default <code>0s</code>. Checks still run during it, but failures do not count toward <code>--health-retries</code> • any single success ends the start period early.
<code>--health-start-interval</code>	<code>docker run --health-start-period=60s --health-start-interval=5s nginx</code>	• How often checks run during the start period, default <code>5s</code>, separate from the steady-state <code>--health-interval</code> • lets a slow-booting app be polled faster while it warms up.
<code>--no-healthcheck</code>	<code>docker run --no-healthcheck redis</code>	Disables any <code>HEALTHCHECK</code> baked into the image, so the container always reports no health status.

TABLE 13

Signal Handling

How Docker delivers Unix signals to your container's PID 1, and why a misplaced shell wrapper or a too-short grace period quietly turns "graceful shutdown" into data corruption. These rows cover the signals Docker actually sends, who receives them, and the flags and form choices that decide whether your app ever gets a chance to clean up.

Concept	Example	Description
SIGTERM	Sent by <code>docker stop</code>	• Graceful shutdown signal delivered to the container's PID 1 • after a grace period (default 10s on Linux, 30s on Windows), Docker follows up with <code>SIGKILL</code> . Tune with <code>-t</code> / <code>--stop-timeout</code> .
SIGKILL	Sent after grace period, or by <code>docker kill</code>	• Force kill signal, kernel-enforced, cannot be caught, blocked, or ignored • immediate termination with no cleanup.
--stop-signal / --stop-timeout	<code>docker run --stop-signal=SIGINT --stop-timeout=5 app</code>	• Overrides the signal <code>docker stop</code> sends (default is the image's <code>STOPSIGNAL</code> , or <code>SIGTERM</code>) and the seconds to wait before <code>SIGKILL</code> • <code>-1</code> waits indefinitely.
SIGINT	Ctrl+C in attached mode	• Interrupt signal sent to PID 1 • most apps treat it like <code>SIGTERM</code> and exit, so Ctrl+C while attached typically stops the container.
SIGHUP	<code>docker kill -s SIGHUP web</code>	• Hangup signal, historically "controlling terminal closed" • many daemons (nginx, HAProxy) repurpose it to reload configuration without restarting.
--signal (docker kill)	<code>docker kill --signal=SIGUSR1 web</code>	• Sends any signal, by name or number, to a running container's PID 1 without stopping it • whether it terminates the container depends on the process, not on <code>docker kill</code> .
--init	<code>docker run --init myapp</code>	• Runs a tiny init (<code>docker-init</code> , backed by tini) as PID 1 instead of your app • reaps zombie processes and forwards signals to your app, useful when your app is a poor PID 1 itself.
Exec form vs shell form	<code>CMD ["app"]</code> vs <code>CMD app</code>	• Exec form (JSON array): the app IS PID 1 and receives signals directly • shell form wraps in <code>/bin/sh -c</code> , sh becomes PID 1 and does not forward signals, so the app never sees <code>SIGTERM</code> .

TABLE 14

Logging Drivers

Docker streams every container's stdout/stderr through a logging driver, and the choice between the default `json-file` driver and alternatives like `local` , `journald` , or remote forwarders shapes disk usage, queryability, and how logs reach a central pipeline.

Concept	Example	Description
json-file (default)	<code>--log-driver json-file</code>	• Stores logs as JSON on disk • queryable with <code>docker logs</code> , no rotation by default, set <code>max-size</code> / <code>max-file</code> to avoid filling <code>/var/lib/docker</code> .
local	<code>--log-driver local</code>	• Optimized local file driver with automatic log rotation and compression by default • Docker's recommended local driver; now supports <code>label</code> , <code>env</code> , and <code>tag</code> log options (Engine 29.5+).
Log rotation (max-size, max-file)	<code>--log-opt max-size=10m</code> <code>--log-opt max-file=3</code>	• <code>log-opts</code> on the <code>json-file</code> driver that cap file size and file count • only applies to newly created containers, not existing ones.
Default logging driver (daemon.json)	<code>{ "log-driver": "local" }</code>	• Sets the daemon-wide default driver in <code>daemon.json</code> • check the active default with <code>docker info --format '{{.LoggingDriver}}'</code> .

Concept	Example	Description
syslog	<code>--log-driver syslog</code> <code>--log-opt syslog-address=udp://host:514</code>	- Forwards logs to a syslog daemon over UDP/TCP/TLS - integrates with centralized logging.
journald	<code>--log-driver journald</code>	- Sends logs to the systemd journal (Linux only) - query with <code>journalctl CONTAINER_NAME=foo</code>.
gelf	<code>--log-driver gelf --log-opt gelf-address=udp://1.2.3.4:12201</code>	- Sends structured Graylog Extended Format messages over TCP/UDP - understood by Graylog, Logstash, and Fluentd.
fluentd	<code>--log-driver fluentd</code> <code>--log-opt fluentd-address=host:24224</code>	- Forwards to a Fluentd collector - popular for Kubernetes and centralized logging.
awslogs	<code>--log-driver awslogs</code> <code>--log-opt awslogs-group=myapp</code>	- Sends logs to AWS CloudWatch Logs - credentials come from the daemon/host (EC2 instance profile or ECS task role), not container env vars.
gcplogs	<code>--log-driver gcplogs</code>	- Forwards to Google Cloud Logging - auto-detects project, zone, and instance from the GCE metadata server.
splunk	<code>--log-driver splunk</code> <code>--log-opt splunk-token=XXX</code> <code>--log-opt splunk-url=https://host:8088</code>	- Sends logs to the Splunk HTTP Event Collector - enterprise logging solution.
Dual logging	<code>docker logs 7d6ac83a89a0</code>	- Docker caches recent output with the <code>local</code> driver so <code>docker logs</code> still works with a remote driver - auto-enables when the configured driver can't read logs back, capped at 5 files of 20MB by default.
Non-blocking mode (mode, max-buffer-size)	<code>--log-opt mode=non-blocking</code> <code>--log-opt max-buffer-size=4m</code>	- Buffers log output per container instead of blocking the app on logging back pressure <code>max-buffer-size</code> sets the buffer (default <code>1m</code>); full buffers drop new messages.
Log tags	<code>--log-opt tag="{{.ImageName}}/{{.Name}}/{{.ID}}"</code>	- Formats the tag identifying a container's log lines - defaults to the first 12 characters of the container ID; supports <code>{{.Name}}</code>, <code>{{.FullID}}</code>, and other template markup.

TABLE 15

Inspect and Debug

Once a container is running, these commands look at it and at the daemon from the outside: low-level JSON via `docker inspect` and its `--format` templates, system-wide views (`events` , `df` , `info` , `version`), the daemon's own logs, and Docker Desktop's `debug` and `init` helpers.

Command	Example	Description
docker inspect	<code>docker inspect --format '{{.State.Status}}' web</code>	- Returns low-level JSON config for any Docker object - pair with <code>--format</code> Go templates to extract one field (canonical scripting pattern).
docker events	<code>docker events --since 1h</code> <code>--filter 'event=oom'</code> <code>--filter 'event=die'</code>	- Continuous stream of daemon lifecycle events (create, start, die, kill, oom, etc.) - chain several <code>--filter</code> flags to isolate one signal, and bound it with <code>--since</code> / <code>--until</code> or it blocks forever.
docker system df	<code>docker system df -v</code>	- Reports Docker-managed disk usage across images, containers, volumes, and build cache <code>-v</code> adds a per-item reclaimable breakdown.
docker system info	<code>docker system info</code>	- Shows runtime configuration: storage driver, cgroup driver and version, kernel version, containerd/runc versions, container counts - check the <code>Cgroup Version</code> line to confirm whether the host runs cgroup v1 or v2 - aliased as <code>docker info</code>.

Command	Example	Description
docker version	<code>docker version</code>	Prints client and server version numbers , the Engine API version , and component versions (containerd, runc, docker-init). Different from <code>docker info</code> .
Read the daemon logs	<code>journalctl -xu docker.service</code>	- Location depends on the OS: <code>journalctl</code> (or <code>/var/log/syslog</code>) on Linux, an <code>init.log</code> under Docker Desktop's data folder on macOS/Windows - set <code>"debug": true</code> in <code>daemon.json</code> for verbose daemon output.
docker debug	<code>docker debug web</code>	- Debug shell with its own toolbox (<code>vim</code>, <code>http</code>, <code>curl</code>, more via <code>install</code> from Nix packages) for any container or image, even ones with no shell - changes never affect the actual image or container.
docker init	<code>docker init</code>	- Scaffolds Dockerfile, compose.yaml, .dockerignore, README.Docker.md for a new project - templates for Go, Node, Python, Rust, Java, PHP, ASP.NET Core.

TABLE 16

Cleanup and Maintenance

Cleanup commands reclaim disk space, but each one targets a different scope and several carry real data-loss risk if used carelessly. Every prune command accepts `--filter`, and most support `until=<time>` and `label=<key>=<value>` (volume prune supports `label` only); knowing what each one does and does not touch, especially `--volumes`, `-a`, and these filters, is the difference between a tidy host and an outage.

Command	Example	Description
docker system prune	<code>docker system prune -a --volumes</code>	- Removes stopped containers, unused networks, dangling images, and unused build cache <code>-a</code> also removes any image not used by a container; volumes are NOT included unless <code>--volumes</code> is passed.
docker image prune	<code>docker image prune -a --filter "until=168h"</code>	- Without <code>-a</code> removes only dangling images (untagged, <code><none></code>) <code>-a</code> removes all unused images; <code>--filter "until=168h"</code> keeps images created in the last 7 days.
docker container prune	<code>docker container prune --filter "label=env=dev"</code>	- Removes all stopped containers and their writable layers - running containers are untouched; <code>--filter</code> narrows by <code>label=</code> or <code>until=</code>; prompts before deletion unless <code>-f</code>.
docker network prune	<code>docker network prune --filter "until=24h"</code>	- Removes every unused user-defined network (no container attached) and prompts for confirmation; <code>-f</code> skips the prompt, <code>--filter "until=24h"</code> narrows by age - the predefined <code>bridge</code>, <code>host</code>, and <code>none</code> networks are never touched.
docker volume prune	<code>docker volume prune --filter "label!=keep"</code>	- By default removes only unused anonymous volumes - use <code>-a</code> / <code>--all</code> to also delete unused named volumes (API 1.42+); <code>--filter</code> supports <code>label=</code> only, no <code>until=</code>.
docker builder prune	<code>docker builder prune --filter "until=24h"</code>	- Clears the BuildKit build cache for the active builder - does not touch images or containers; <code>--keep-storage</code> caps how much cache stays; next build rebuilds layers from scratch.

TABLE 17

Docker Compose Services

How each `services:` entry is configured: the attributes that decide what runs, how it reaches the network, what it depends on, and how the platform treats it. Most startup-order, port, and resource bugs in Compose stacks trace back to misreading one of these keys.

Attribute	Example	Description
image	<code>image: nginx:alpine</code>	- Names the image to run - pulled from the registry if not present locally.

Attribute	Example	Description
build	<pre>build: context: . dockerfile: Dockerfile.prod</pre>	- Builds an image from a Dockerfile <code>context</code> is resolved relative to the Compose file's directory, not the shell's cwd.
ports	<pre>ports: - "8080:80"</pre>	- Publishes ports as "HOST:CONTAINER" always quote short syntax: YAML parses unquoted <code>xx:yy</code> as a base-60 number.
expose	<pre>expose: - "3000" - "8080-8085/tcp"</pre>	- Documents container ports reachable by other services on the same network - unlike <code>ports</code>, nothing here is published to the host.
volumes	<pre>volumes: - ./data:/data:ro</pre>	- Mounts volumes or bind mounts - trailing <code>:ro</code> mounts the path read-only inside the container.
environment	<pre>environment: NODE_ENV: production</pre>	- Sets runtime environment variables - Compose's own interpolation expands <code>\${VAR}</code> and unbraced <code>\$VAR</code> from the shell or <code>.env</code> - escape a literal <code>\$</code> with <code>\$\$</code> to stop it from being interpolated.
depends_on	<pre>depends_on: db: condition: service_healthy restart: true required: false</pre>	- Controls startup order; default <code>service_started</code> only waits for the container to start <code>service_healthy</code> waits for the dependency's healthcheck to pass <code>restart: true</code> restarts the dependent if the dependency restarts; <code>required: false</code> allows startup even if the dependency is unavailable.
service_started	<pre>depends_on: db: condition: service_started</pre>	- Default behavior: waits only for the dependency container to be started - does not wait for the app inside to be ready, so first connections can still fail.
service_healthy	<pre>depends_on: db: condition: service_healthy</pre>	- Waits for the dependency's healthcheck to report healthy - requires a <code>HEALTHCHECK</code> in the image or a <code>healthcheck:</code> block; without one, the dependent never starts.
service_completed_successfully	<pre>depends_on: migrate: condition: service_completed_successfully</pre>	- Waits for a one-off dependency to exit with code 0 - canonical "run migrations first" pattern; non-zero exit blocks the dependent.
networks	<pre>networks: - frontend</pre>	- Attaches the service to named networks - inside a shared network, services reach each other by service name on the container port.
restart	<pre>restart: unless-stopped</pre>	- Sets the container restart policy - options: <code>no</code> (default), <code>always</code>, <code>on-failure[:N]</code>, <code>unless-stopped</code>.
command	<pre>command: npm run dev</pre>	Overrides the image's CMD only - the image's <code>ENTRYPOINT</code> still runs and receives this as its arguments.
entrypoint	<pre>entrypoint: /app/entrypoint.sh</pre>	Replaces the image's ENTRYPOINT entirely - when set, Compose also ignores the image's <code>CMD</code> instead of passing it as arguments, unlike overriding entrypoint with plain <code>docker run</code>.
healthcheck	<pre>healthcheck: test: ["CMD", "curl", "-f", "http://localhost/"] interval: 30s start_interval: 2s</pre>	- Defines a container health probe <code>CMD</code> execs the args directly; <code>CMD-SHELL</code> wraps in <code>/bin/sh -c</code> so operators like <code> </code> work <code>start_interval</code> sets a faster check rate during <code>start_period</code> so dependent services start sooner.
stop_grace_period	<pre>stop_grace_period: 30s</pre>	- How long Compose waits after the stop signal (default SIGTERM) before sending SIGKILL - default is 10 seconds.

Attribute	Example	Description
deploy	<pre>deploy: resources: limits: memory: 512M</pre>	- Most keys (<code>replicas</code> , <code>placement</code> , <code>update_config</code>) are Swarm-only and ignored by <code>docker compose up</code> <code>deploy.resources</code> for limits and GPU reservations does apply to plain Compose.
sysctls	<pre>sysctls: net.core.somaxconn: 1024</pre>	- Sets kernel parameters inside the container - only sysctls namespaced in the kernel are allowed; Compose refuses one that would also change the host, so a host-wide setting like an inotify limit cannot be set here.

TABLE 18

Docker Compose Commands

The Compose CLI manages multi-container apps defined in a `compose.yaml` file as a single **Compose project**, a scope Compose tracks by project name. The legacy `docker-compose` (v1, hyphenated) was removed from Docker Desktop in September 2023 (v4.23.0); only `docker compose` (space, no hyphen) ships today. Knowing what each command actually mutates (containers, networks, volumes, images) is the difference between a clean restart and accidentally deleting your database.

Command	Example	Description
docker compose up	<pre>docker compose up -d --wait</pre>	Builds, (re)creates, starts all services and their networks <code>-d</code> detaches; <code>--wait</code> blocks until services report running/healthy; recreates containers whose config changed since last <code>up</code> .
docker compose down	<pre>docker compose down -v</pre>	- Stops and removes containers and the project's networks - Keeps named volumes by default; <code>-v</code> also removes named volumes (data loss).
docker compose ps	<pre>docker compose ps</pre>	- Lists running containers for the current Compose project - add <code>-a</code> for stopped ones.
docker compose logs	<pre>docker compose logs -f web</pre>	- Shows aggregated logs from services <code>-f</code> follows; pass a service name to filter to one.
docker compose exec	<pre>docker compose exec web sh</pre>	- Runs a command inside a running service container - allocates a TTY by default; <code>--index</code> picks a replica.
docker compose build	<pre>docker compose build --no-cache</pre>	Builds or rebuilds service images from each service's <code>build</code> : <code>--no-cache</code> discards the build cache; pair with <code>--pull</code> to refresh base images.
docker compose pull	<pre>docker compose pull</pre>	- Pulls images for services from their registries does not start containers.
docker compose restart	<pre>docker compose restart web</pre>	- Restarts existing containers without re-reading the Compose file - config edits do not apply until you run <code>up -d</code> instead.
docker compose stop	<pre>docker compose stop</pre>	- Stops services without removing containers, networks, or volumes - faster than <code>down</code> ; resume with <code>start</code> .
docker compose start	<pre>docker compose start</pre>	Starts previously stopped containers for the project - never creates new containers, use <code>up</code> for that.
docker compose run	<pre>docker compose run --rm web npm test</pre>	- Runs a one-off command in a new container based on the service <code>--rm</code> removes it after exit; starts linked services unless <code>--no-deps</code> .
docker compose config	<pre>docker compose config --services</pre>	- Renders the merged, resolved model Compose would apply: combines <code>-f</code> files, resolves variables, expands short-notation <code>--services</code> or <code>--images</code> list just those names, useful for debugging what will actually run.
docker compose -f / -p	<pre>docker compose -f compose.yaml -f compose.prod.yaml -p myapp up -d</pre>	<code>-f</code> supplies one or more Compose files; later files override and add to earlier ones <code>-p</code> sets the project name that scopes containers, networks and volumes.

Command	Example	Description
docker compose watch	<code>docker compose watch</code>	- Reacts to file changes declared under <code>develop.watch</code> - actions: <code>sync</code>, <code>rebuild</code>, <code>sync+restart</code> (modern replacement for <code>bind-mount-and-restart</code>).

TABLE 19

Docker Compose Advanced Features

Compose's advanced primitives go beyond a basic `compose.yaml`: profiles toggle optional services per environment, `extends` / `include` / `override` files compose larger stacks out of smaller pieces, `secrets` and `configs` mount files instead of leaking env vars, `develop.watch` keeps containers in sync with source changes, and the `models` top-level element declares AI/ML model dependencies alongside services.

Feature	Example	Description
profiles	<code>profiles: [debug]</code>	- Groups services for selective activation <code>docker compose --profile debug up</code> only starts debug services.
env_file	<code>env_file:</code> <code>- .env.prod</code>	- Loads environment variables from files - keeps secrets out of Compose file.
override files	<code>compose.override.yaml</code>	- Automatically merged with <code>compose.yaml</code> environment-specific overrides without modifying the base file.
Interpolation defaults	<code>COMPOSE_DEBUG=\${DEV_MODE:-false}</code> <code>DB_PASSWORD=\${DB_PASSWORD:?error}</code>	<code>\${VAR:-default}</code> substitutes a fallback when a variable is unset or empty <code>\${VAR:?error}</code> exits with an error instead of silently using an empty value.
COMPOSE_* environment variables	<code>COMPOSE_FILE=compose.yaml:compose.prod.yaml</code> <code>COMPOSE_PROJECT_NAME=myapp</code>	- Pre-defined variables that configure Compose itself, set via <code>.env</code> file, shell, or <code>--env</code> flag <code>COMPOSE_FILE</code> lists which files to use, <code>COMPOSE_PROJECT_NAME</code> sets the container name prefix.
secrets	<code>secrets:</code> <code>- db_password</code>	- Injects secrets as files in container - mounted at <code>/run/secrets/</code>.
configs	<code>configs:</code> <code>- nginx.conf</code>	- Like secrets but for non-sensitive config files - also mounted as files.
extends	<code>extends:</code> <code>file: common.yaml</code> <code>service: app</code>	Inherits configuration from another service - enables reusable base definitions.
include	<code>include:</code> <code>- monitoring.yaml</code> <code>- oci://docker.io/org/stack:latest</code>	- Includes separate Compose files or OCI artifacts to combine projects - update to v2.40.2+ if using OCI includes, CVE-2025-62725 was a path-traversal fix.
develop (watch)	<code>develop:</code> <code>watch:</code> <code>- action: sync</code> <code>path: ./src</code> <code>target: /app/src</code> <code>- action: rebuild</code> <code>path: package.json</code>	- Configures hot-reload for development (formerly <code>x-develop</code>, now without the <code>x-</code> prefix) - actions: <code>sync</code> (copy files), <code>rebuild</code> (rebuild image), <code>sync+restart</code> (copy and restart) <code>initial_sync</code> syncs all matching files immediately on <code>docker compose watch</code> start.
Lifecycle hooks (pre_start/post_start/pre_stop)	<code>services:</code> <code>app:</code> <code>depends_on:</code> <code>db:</code> <code>condition: service_healthy</code> <code>pre_start:</code> <code>- command: ["/manage.py", "migrate"]</code>	<code>pre_start</code> (Compose 5.3+) runs ephemeral init containers to completion, in order, before the service container starts, failing the bring-up on a non-zero exit <code>post_start</code> / <code>pre_stop</code> instead run a command inside the already-running service container.

Feature	Example	Description
YAML anchors	<pre>x-common: &common restart: always services: web: <<: *common</pre>	- Reuses config blocks with anchors and aliases <code>x-</code> prefix defines extension fields that Compose ignores.
GPU support	<pre>deploy: resources: reservations: devices: - driver: nvidia count: 1 capabilities: [gpu]</pre>	- Requests GPU access for a service (NVIDIA and AMD supported) - requires NVIDIA Container Toolkit or AMD ROCm driver on host.
models (Compose AI)	<pre>models: llm: model: ai/smollm2 context_size: 4096</pre>	- Top-level element declaring AI/ML model dependencies as OCI artifacts alongside services - platform (e.g. Docker Model Runner) injects <code><NAME>_URL</code> and <code><NAME>_MODEL</code> env vars into consuming services; portable across cloud platforms.

TABLE 20

Security Practices

Container security is layered defense: drop privileges, shrink the attack surface, and constrain what the kernel will let the container do. These practices stack: running as non-root, dropping capabilities, applying a seccomp profile, and remapping user namespaces each close a different escape path, and together they are what stands between a single application bug and host compromise.

Practice	Example	Description
Run as non-root user	<code>USER node</code>	- Creates and switches to non-root user - prevents privilege escalation attacks.
Read-only root filesystem	<code>docker run --read-only nginx</code>	- Makes container filesystem immutable - prevents tampering; use tmpfs for writable dirs.
Drop capabilities	<code>docker run --cap-drop ALL --cap-add NET_BIND_SERVICE nginx</code>	- Removes Linux capabilities - principle of least privilege: drop ALL, then add only what's needed.
No privileged mode	Avoid <code>--privileged</code>	Never use in production - gives full host access, defeats containerization.
Use secrets	<code>RUN --mount=type=secret,id=token</code>	- Never hardcode secrets in images - use BuildKit secrets or runtime secrets.
Scan images	<code>docker scout cves nginx:latest</code>	- Scans for vulnerabilities - integrates with Docker Scout, Trivy, Snyk.
Default seccomp profile	<code>docker run --security-opt seccomp=builtin nginx</code>	- Docker applies a built-in seccomp profile to every container automatically - blocks around 44 syscalls out of 300+ unless overridden.
Security profiles	<code>--security-opt seccomp=profile.json</code>	- Apply custom seccomp/AppArmor/SELinux profiles - restricts syscalls and actions beyond the default.
--no-new-privileges	<code>docker run --security-opt no-new-privileges nginx</code>	- Prevents processes from gaining additional privileges via setuid/setgid - critical for defense in depth.
Protect the Docker daemon socket	<code>dockerd --tlsverify --tlscacert=ca.pem --tlscert=server-cert.pem --tlskey=server-key.pem -H=0.0.0.0:2376</code>	- The daemon defaults to a local, non-networked socket - exposing it remotely without TLS or SSH grants root access on the host.

Practice	Example	Description
Rootless mode	<code>dockerd-rootless-setup tool.sh install</code>	- Runs Docker daemon without root privileges - mitigates potential vulnerabilities in daemon and container runtime; Engine 29.5+ uses <code>gvisor-tap-vsock</code> as the default rootless network driver.
User namespace remapping	<code>dockerd --userns-remap="testuser:testuser"</code>	- Maps a container's root user to an unprivileged host user - limits damage from a container breakout; set daemon-wide via <code>daemon.json</code> .
Content trust (docker trust)	<code>docker trust sign myapp:latest</code>	- Verifies image publisher and integrity with digital signatures before push/pull being retired: the Notary v1 service behind it shuts down December 2026.

TABLE 21

Distroless and Minimal Images

Choosing a smaller base image shrinks the attack surface, the CVE footprint, and the pull/startup time, but each minimal base trades away something. This table compares the common minimal-base options so you can pick by what your app actually needs at runtime (a shell, a libc, a package manager, none of the above).

Image	Example	Description
Minimal base images	<code>FROM alpine:3.21</code> <code>FROM gcr.io/distroless/static</code> <code>FROM scratch</code>	- Choosing a minimal base image cuts image size, download time and CVE exposure to only what the app needs at runtime - alpine, distroless and scratch below cover the common choices, each trading away something different
Alpine Linux	<code>FROM alpine:3.21</code>	- Lightweight ~5 MB base - uses musl libc (not glibc), so prebuilt Python/Node binary wheels and other glibc-linked binaries may fail or need recompiling.
Distroless	<code>FROM gcr.io/distroless/static-debian12</code>	No shell, no package manager, only your app and its runtime dependencies <code>static</code> is about 2 MB, roughly half alpine's size, cutting attack surface and CVE noise
Distroless :debug variant	<code>FROM gcr.io/distroless/base-debian12:debug</code>	- Adds a busybox shell on top of distroless so you can <code>docker run --entrypoint=sh</code> - intended for local debugging, never production.
Scratch	<code>FROM scratch</code>	- Completely empty base: no libc, no shell, no files - works only for statically linked binaries (e.g. Go with <code>CGO_ENABLED=0</code> , Rust with a musl target)
Chiseled Ubuntu images	<code>FROM ubuntu/jre:25-26.04_stable</code>	- Canonical's chisel slices a package down to just the files an app needs, still vendor-supported like classic Ubuntu - a <code>-slim</code> official-image tag (e.g. <code>python:3.13-slim</code>) gets a similar effect by dropping the default tag's extra packages
Docker Hardened Images (DHI)	<code>FROM dhi.io/python:3.13</code>	- Docker's hardened, near-zero-CVE images on Alpine or Debian foundations, pulled from the <code>dhi.io</code> registry - free and open source under Apache 2.0, ships with SBOMs and SLSA provenance attestations

TABLE 22

Docker DHI CLI

The `docker dhi` CLI plugin (Docker Desktop 4.65+) manages Docker Hardened Images: browse the catalog, view SBOM/provenance attestations, mirror images into your organization's registry, and create customized variants that add your own packages on top of a DHI base. A standalone `dhictl` binary is also available for headless CI environments.

Command	Example	Description
docker dhi catalog list	<code>docker dhi catalog list --filter golang</code>	- Lists available DHI images - filter by name, <code>--fips</code>, or <code>--stig</code>.
docker dhi catalog get	<code>docker dhi catalog get golang</code>	Gets detailed information about one DHI image or Helm chart, including available tags and CVE counts.
docker dhi attestation sbom	<code>docker dhi attestation sbom dhi/nginx:1.27</code>	Displays the SPDX SBOM attached to a DHI image as an OCI attestation.
docker dhi attestation list	<code>docker dhi attestation list dhi/nginx:1.27</code>	Lists every attestation attached to an image, retrieved via the OCI Referrers API, including provenance.
docker dhi mirror start	<code>docker dhi mirror start --org my-org dhi/golang,my-org/dhi-golang</code>	- Begins automatically mirroring a DHI image into your Docker Hub organization registry <code>--dependencies</code> also mirrors base layers; Docker patches the mirror on its SLA.
docker dhi mirror list	<code>docker dhi mirror list --org my-org</code>	Lists all DHI images currently mirrored to your organization with their status.
docker dhi mirror stop	<code>docker dhi mirror stop dhi-python --org my-org</code>	- Stops mirroring one or more repositories - add <code>--delete</code> to also remove them.
docker dhi customization prepare	<code>docker dhi customization prepare golang 1.25 --org my-org --destination my-org/dhi-golang > my-customization.yaml</code>	- Generates a YAML scaffold for creating a custom variant of a DHI image - edit the YAML to add packages or configuration, then <code>docker dhi customization create</code>.
docker dhi customization create	<code>docker dhi customization create my-customization.yaml --org my-org</code>	- Submits the customization YAML to Docker to build a custom DHI variant and push it to your org registry <code>docker dhi customization build logs <id> <build-id></code> streams the build output.
docker dhi customization list	<code>docker dhi customization list --org my-org</code>	Lists all Docker Hardened Images customizations, filterable by name, repository or source.
docker dhi auth apk/deb	<code>docker dhi auth apk</code>	- Generates enterprise package repository credentials for installing Docker-patched packages in your own images <code>apk</code> for Alpine-based images, <code>deb</code> for Debian-based images.
dhictl	<code>dhictl mirror start --org my-org -r dhi/golang,my-org/dhi-golang</code>	Standalone binary form of the DHI CLI for headless or CI environments where the Docker CLI plugin isn't installed.

TABLE 23

Docker Scout Commands

Docker Scout is Docker's built-in supply-chain security tool: it reads an image's SBOM (generating one if missing), looks up known CVEs against it, and suggests safer base images. These CLI commands are the day-to-day surface, triaging vulnerabilities, comparing image versions, and enrolling an organization for policy enforcement.

Command	Example	Description
docker scout cves	<code>docker scout cves nginx:latest</code>	- Scans an image for CVEs - shows severity, CVSS score, affected package, and fix availability.
docker scout quickview	<code>docker scout quickview nginx:latest</code>	- Displays a summary of vulnerabilities and policy compliance - fast overview without printing the full CVE list.
docker scout recommendations	<code>docker scout recommendations nginx:latest</code>	- Suggests base image refresh or update - shows benefits like fewer CVEs and smaller image size for each alternative.
docker scout sbom	<code>docker scout sbom nginx:latest</code>	- Generates a Software Bill of Materials - lists every package and version (SPDX or CycloneDX output adds licenses and relationships).
docker scout compare	<code>docker scout compare nginx:1.25 --to nginx:latest</code>	Compares vulnerability profiles of two image versions - canonical way to verify a base image bump actually reduced CVEs.
docker scout attestation add	<code>docker scout attestation add --file sbom.json nginx:latest</code>	- Attaches an attestation (SBOM, provenance, or other predicate) to an image <code>attestation get</code> and <code>attestation list</code> retrieve or enumerate them later (experimental).
docker scout environment	<code>docker scout environment staging nginx:latest</code>	- Names an environment (staging, production) and records an image to it - lets <code>docker scout compare --to-env production</code> track drift against that baseline (experimental).
docker scout policy	<code>docker scout policy nginx:latest --org myorg</code>	- Evaluates Rego policies against an image and reports pass or violation results (experimental) <code>-e/--exit-code</code> fails a pipeline when a policy isn't met.
docker scout enroll	<code>docker scout enroll myorg</code>	Enrolls an organization in Docker Scout - required before policy enforcement, repository integration, and team dashboards work for org images.

TABLE 24

Context and Remote Docker

A Docker context bundles the endpoint URL, TLS material, and name of one daemon so a single `docker` CLI can manage many daemons, local Docker Desktop, an SSH-reachable production server, an ECS endpoint, and switch between them. SSH-based contexts are the canonical secure pattern for remote control because they reuse your SSH keys and avoid exposing the Docker daemon socket on a public TCP port.

Command	Example	Description
docker context create	<code>docker context create prod --docker "host=ssh://user@host"</code>	- Registers a named connection to a Docker daemon (local, <code>tcp://</code>, or <code>ssh://</code>) - metadata is stored under <code>~/.docker/contexts/</code>.
docker context use	<code>docker context use prod</code>	- Sets the active context in <code>~/.docker/config.json</code> - persists across shells until changed or overridden by <code>DOCKER_HOST</code> / <code>DOCKER_CONTEXT</code> / <code>--context</code>.
docker context ls	<code>docker context ls</code>	- Lists every context with its <code>DOCKER_ENDPOINT</code> URL - the active one is marked with *.
docker context inspect	<code>docker context inspect prod</code>	- Prints a context's endpoint, TLS material, and metadata as JSON <code>-f/--format</code> renders a Go template instead.

Command	Example	Description
docker context rm	<code>docker context rm prod</code>	- Removes a context fails on the active context unless you pass <code>-f / --force</code> or <code>docker context use default</code> first.
--context flag	<code>docker --context prod ps</code>	- Per-command override of which daemon a single invocation talks to highest priority: beats <code>DOCKER_HOST</code> and the active context set by <code>docker context use</code>.
DOCKER_HOST variable	<pre>export DOCKER_HOST=ssh://user@host</pre>	- Per-shell daemon-socket override beats the active context but is itself overridden by <code>DOCKER_CONTEXT</code> and <code>--host / --context</code>.

TABLE 25

Daemon and Engine Configuration

Most of these settings live in `/etc/docker/daemon.json` (or an equivalent `dockerd` flag) and take effect only after a daemon restart or `systemctl reload docker`. They shape the daemon itself, its storage, networking defaults and exposed endpoints, rather than any single container.

Concept	Example	Description
daemon.json	<pre>/etc/docker/daemon.json ~/.config/docker/daemon.json # rootless \$ dockerd --config-file /path/to/daemon.json</pre>	- JSON file the daemon reads on startup, one key per setting - the same option can't be set both as a flag and in the file, or the daemon refuses to start
data-root	<pre>{ "data-root": "/mnt/docker-data" }</pre>	- Relocates the daemon's storage directory (default <code>/var/lib/docker</code>) - doesn't move containerd's own data when the containerd image store is enabled
Storage drivers (overlay2 and the rest)	<pre>\$ docker info Storage Driver: overlay2</pre>	<code>overlay2</code> is the default classic driver on every supported Linux distribution <code>fuse-overlayfs</code>, <code>btrfs</code>, <code>zfs</code> and <code>vfs</code> cover rootless or advanced cases
containerd image store	<pre>{ "features": { "containerd-snapshotter": true } }</pre>	- Default storage backend on fresh Docker Engine 29+ installs - uses containerd snapshotters instead of classic graph drivers, enabling multi-platform images and attestations
Live restore	<pre>{ "live-restore": true }</pre>	- Keeps containers running when the daemon stops, crashes or upgrades - guaranteed only across patch releases, not major version upgrades
Remote access over TCP with TLS	<pre>\$ dockerd --tls=true --tlscert=/var/docker/server.pem --tlskey=/var/docker/serverkey.pem --host tcp://192.168.59.3:2376</pre>	- Exposes the daemon socket over TCP for remote clients unauthenticated TCP is discouraged and will need explicit opt-in in a future release
Daemon proxy configuration	<pre>{ "proxies": { "http-proxy": "http://proxy.example.com:3128", "no-proxy": "*.test.example.com,example.org" } }</pre>	- Routes the daemon's own pulls and pushes through a proxy <code>daemon.json</code> settings take precedence over <code>HTTP_PROXY</code> / <code>HTTPS_PROXY</code> environment variables

Concept	Example	Description
default-address-pools	<pre>{ "default-address-pools": [{ "base": "172.17.0.0/16", "size": 24 }] }</pre>	Sets the subnet pool Docker draws from when auto-assigning a network's address range, avoiding conflicts with the host's own network
IPv6 for the default bridge network	<pre>{ "ipv6": true, "fixed-cidr-v6": "2001:db8:1::/64" }</pre>	• <code>ipv6</code> turns on IPv6 for the default bridge network • <code>fixed-cidr-v6</code> assigns it a subnet for dynamic address allocation
Alternative container runtimes (--runtime)	<pre>{ "runtimes": { "gvisor": { "runtimeType": "io.containerd.runsc.v1" } } } \$ docker run --runtime gvisor hello-world</pre>	• Registers a non-default OCI or containerd-shim runtime (gVisor, Kata, youki) with the daemon • <code>runc</code> remains the default
Prometheus metrics endpoint	<pre>{ "metrics-addr": "127.0.0.1:9323" }</pre>	• Exposes Docker's own Prometheus-compatible metrics on the given address • binding <code>0.0.0.0</code> puts the port on the network, not just loopback
default-stop-timeout (Engine 29.7.0)	<pre>{ "default-stop-timeout": 30 }</pre>	Daemon-wide fallback stop timeout (seconds) applied to a container that sets none of its own
Embedded containerd (experimental)	<pre>{ "features": { "embedded-containerd": true } }</pre>	• Runs containerd inside the daemon process, reached over an in-memory connection instead of a separate managed process • experimental (Engine 29.7.0); doesn't support the container runtime interface (CRI)

TABLE 26

Docker Swarm Basics

Swarm mode is the orchestrator built directly into the Docker Engine, no separate install needed. One `docker swarm init` turns the current engine into a single-node cluster (a manager), and additional engines join as workers or managers via a join token. From there you create **services** (replicated containers spread across the cluster) and **stacks** (a whole Compose file deployed as a unit), and the managers schedule tasks, run rolling updates, and keep the desired replica count alive on the routing mesh.

Command	Example	Description
docker swarm init	<code>docker swarm init</code>	• Turns the current Docker Engine into a single-node Swarm cluster • the current node becomes the leader manager.
docker swarm join	<code>docker swarm join --token TOKEN host:2377</code>	• Joins a node to an existing Swarm • the token used (worker or manager) determines the role of the joining node.
docker service create	<code>docker service create --replicas 3 --name web nginx</code>	• Creates a new Swarm service distributed across the cluster • <code>--replicas N</code> declares the desired task count the schedulers keep alive.
docker service scale	<code>docker service scale web=5</code>	• Scales an already-running replicated service to a new replica count • cannot be used on <code>global</code> -mode services.
docker service update	<code>docker service update --image nginx:1.21 web</code>	• Applies a rolling update, by default updating 1 task at a time, controlled by <code>--update-parallelism</code> and <code>--update-delay</code> • supports <code>--rollback</code> on failure.
docker service ps	<code>docker service ps redis</code>	Lists the tasks running as part of a service, with each task's node, current and desired state .

Command	Example	Description
docker service logs	<code>docker service logs -f web</code>	- Batch-retrieves the logs of a service or a single task <code>-f / --follow</code> streams new output; only works when the service uses the <code>json-file</code> or <code>journald</code> logging driver.
docker node ls	<code>docker node ls</code>	- Lists every node the Swarm manager knows about, with status, availability, and manager status - only runs on a manager node.
docker secret create	<pre>printf "my-secret-pw" docker secret create my_secret -</pre>	Creates a Swarm secret from standard input or a file, made available to services that reference it.
docker stack deploy	<code>docker stack deploy -c compose.yml myapp</code>	- Deploys a Compose file (v3.0+) as a Swarm stack, creating its services, networks, and volumes <code>build:</code> is ignored: images must already exist in a registry.

TABLE 27

Docker Build Cloud and Docker Offload

Docker Build Cloud swaps in a managed, shared BuildKit builder in the cloud for `docker buildx build`, same Dockerfile, same command, just faster and natively multi-platform. Docker Offload goes further and runs whole containers on a full remote Docker daemon over an SSH tunnel, for machines that cannot run containers locally at all.

Feature	Example	Description
Connect to cloud builder	<code>docker buildx create --driver cloud myorg/mybuilder</code>	Registers a local endpoint for an existing cloud builder (the builder itself is provisioned in the Docker Build Cloud Dashboard) - the local endpoint is named <code>cloud-myorg-mybuilder</code>.
Build with cloud	<code>docker buildx build --builder cloud-myorg-mybuilder -t myapp .</code>	- Sends the build context to the remote builder - build runs on AWS EC2 with more CPU/RAM than the laptop, then result is loaded back or pushed to a registry.
docker offload start	<code>docker offload start --gpu</code>	- Connects Docker Desktop to a remote Docker daemon over a secure SSH tunnel, so <code>docker run</code> and <code>docker build</code> both execute in the cloud - requires a Docker Business + Offload subscription; <code>--gpu</code> requests a GPU-enabled remote engine.
Set as default builder	<code>docker buildx use cloud-myorg-mybuilder --global</code>	- Sets the cloud builder as the default for <code>docker buildx build</code> <code>--global</code> persists the selection across shells; affects <code>docker buildx build</code> only, not the legacy <code>docker build</code>.
List builders	<code>docker buildx ls</code>	- Lists all builder instances with their driver, endpoint, status, BuildKit version, and supported platforms - the current default is marked with a <code>*</code>.
Shared build cache	<code>docker buildx du</code> <code>docker buildx prune</code>	- A cloud builder's cache is shared by every teammate using it and garbage-collected automatically <code>du</code> checks the current cache state; <code>prune</code> clears it manually and also clears it for the rest of the team.
Native multi-platform builds	<code>docker buildx build --builder cloud-myorg-mybuilder --platform linux/amd64,linux/arm64 -t myapp --push .</code>	- The cloud builder has native <code>linux/amd64</code> and <code>linux/arm64</code> nodes - each platform runs on its real CPU in parallel with no QEMU emulation overhead, typically much faster than emulated cross-builds on a single-arch laptop or CI runner.
Build Cloud in CI	<pre>- uses: docker/setup-buildx-action@v4 with: driver: cloud endpoint: myorg/mybuilder</pre>	- Point any CI platform's Buildx setup step at <code>driver: cloud</code> and the builder's <code>endpoint</code> to run pipeline builds on Build Cloud instead of a local runner - the same pattern works in GitHub Actions, GitLab, CircleCI, Jenkins and other CI systems.

Feature	Example	Description
docker offload stop / status	<code>docker offload status</code> <code>docker offload stop</code>	<code>status</code> reports whether the session is idle, connecting, or running and its duration <code>stop</code> tears the remote environment down; an idle session auto-terminates after 5 minutes, so push images to a registry first to keep them.

TABLE 28

Docker Model Runner

Docker Model Runner (DMR) is a Docker Desktop / Docker Engine feature for pulling, running, and serving local AI models with a `docker model` CLI that mirrors familiar image commands. Models are stored as OCI artifacts separately from container images, run on a host-native inference server (llama.cpp by default), and are reachable over OpenAI-, Anthropic-, and Ollama-compatible HTTP APIs. As of Docker Desktop 4.71, DMR is **disabled by default** and must be explicitly enabled in Settings.

Command	Example	Description
Enable Model Runner	<code>docker desktop enable model-runner</code>	- Turns on Model Runner in Docker Desktop from the CLI <code>--tcp</code> exposes the host-side REST API on a port (default 12434); <code>--cors</code> sets allowed origins.
docker model pull	<code>docker model pull ai/llama3.2</code>	- Downloads a model from Docker Hub, an OCI registry, or Hugging Face - stored locally as OCI artifacts in a separate model store (not in <code>docker images</code>).
docker model run	<code>docker model run ai/llama3.2</code>	- Runs a one-time prompt or starts an interactive chat in the terminal - the model is loaded on demand and unloaded after ~5 min idle.
docker model run --detach	<code>docker model run -d ai/llama3.2</code>	Pre-loads the model into memory without an interactive session - the OpenAI-compatible REST API is always served by Model Runner: <code>-d</code> just warms the cache.
docker model list	<code>docker model list</code>	Lists locally pulled models with size, parameters, and quantization (alias: <code>docker model ls</code>).
docker model rm	<code>docker model rm ai/llama3.2</code>	Removes a model from the local model store to reclaim disk space.
docker model status	<code>docker model status</code>	Reports whether Model Runner is running and which inference engines are currently active.
Inference engines	<code>docker model install-runner --backend vllm --gpu cuda</code>	- Beyond the default llama.cpp, vLLM (Safetensors, NVIDIA GPU, high throughput) and Diffusers (image generation) run as installable backends - select one per run with <code>--backend</code> or a model's <code>-vllm</code> suffix.
docker model package	<code>docker model package --gguf ./model.gguf --push myorg/mymodel:Q4_K_M</code>	- Packages a local GGUF, Safetensors, or DDUF file into a Docker Model OCI artifact - add <code>--push</code> to publish it straight to a registry instead of the local store.
docker model push	<code>docker model push myorg/llama3:8b</code>	Publishes an already-packaged local model to Docker Hub or Hugging Face.
OpenAI-compatible API	<code>curl http://model-runner.docker.internal/engines/v1/chat/completions</code>	- Reach the running model via OpenAI-, Anthropic-, or Ollama-compatible endpoints (including <code>/responses</code> for the OpenAI Responses API) - from containers use <code>model-runner.docker.internal</code>; from the host use <code>http://localhost:12434/engines/v1</code>.

TABLE 29

Docker AI (Gordon)

Gordon is Docker's AI-powered assistant available from Docker Desktop 4.74+ and the `docker ai` CLI. It reads your live Docker environment, running containers, logs, images, Compose files, proposes actions, and executes them only after your approval. Gordon on Docker Hub and docs.docker.com is free with no Docker account required; CLI and Docker Desktop usage counts against a usage plan.

Command	Example	Description
docker ai (TUI)	<code>docker ai</code>	- Opens an interactive terminal UI for Gordon - type questions or tasks; Gordon proposes actions and waits for <code>y</code> approval before executing.
docker ai (one-shot)	<code>docker ai "show me logs from my nginx container"</code>	- Runs a one-off prompt without the full TUI - Gordon reads the environment, proposes relevant commands (e.g. <code>docker logs</code>), and runs them on approval.
Gordon in Docker Desktop	(Docker Desktop sidebar → Gordon)	- Same capabilities as CLI Gordon but with a graphical session sidebar - lets you link a project directory and run queries with richer visual output.
Permissions and approval	<code>docker ai "clean up stopped containers"</code> Approve? [Y/n/a]: a	- Shell commands, file writes, and web fetches all need <code>y / n / a</code> approval before Gordon runs them (reading files and searching docs does not) YOLO mode turns off approval checks entirely.
Dockerfile review	<code>docker ai "review my Dockerfile for best practices"</code>	- Analyzes the Dockerfile in the current directory and suggests optimizations: smaller base, layer order, cache hits, secret handling - Gordon can apply the edits in-place.
Debug container failures	<code>docker ai "why did my db container crash"</code>	- Reads container logs and inspect output, diagnoses the root cause, and suggests a fix - any proposed <code>docker</code> commands shown before execution.
Configure Gordon's tools	Allow: cat, grep, ls Deny: chown, chmod	- Enables or disables each of Gordon's built-in tools (shell, filesystem, web) and sets fine-grained command allow/deny-lists - Business orgs can lock tool settings for every user.

TABLE 30

Docker MCP Toolkit

The Docker MCP Toolkit (`docker mcp`) installs, runs and connects containerized Model Context Protocol (MCP) servers, such as GitHub, Playwright and Grafana, to AI clients including Claude Desktop, Cursor, VS Code and Gordon. The catalog now lists 300+ servers, and each one runs in its own isolated container under Docker's security model. Requires Docker Desktop 4.62 or later.

Command	Example	Description
docker mcp profile create	<code>docker mcp profile create --name web-dev</code>	- Creates a named profile, a workspace that organizes a set of MCP servers - profiles are the unit of sharing, pushed and pulled with <code>docker mcp profile push/pull <oci-ref></code>
docker mcp profile server add	<code>docker mcp profile server add web-dev --server catalog://mcp/docker-mcp-catalog/github-official</code>	- Adds a server to a profile using a catalog URI (<code>catalog://</code>), Docker image URI (<code>docker://</code>), local YAML (<code>file://</code>), or HTTPS registry URL - servers requiring OAuth show "Configuration Required" in Docker Desktop.
docker mcp gateway run	<code>docker mcp gateway run --profile web-dev</code>	- Starts the MCP Gateway over <code>stdio</code> with the given profile's servers - used to connect any AI client manually via its MCP JSON config (<code>"command": "docker", "args": ["mcp", "gateway", "run", "--profile", "web-dev"]</code>).
docker mcp client connect	<code>docker mcp client connect vscode --profile my-project</code>	- Connects a named supported client (claude, cursor, vscode, etc.) to a profile - writes the appropriate client-specific config file (e.g. <code>.vscode/mcp.json</code>).

Command	Example	Description
docker mcp catalog server ls	<code>docker mcp catalog server ls mcp/docker-mcp-catalog</code>	- Lists all servers in the Docker MCP Catalog (or a custom catalog) by name and ID - the Server ID is used in <code>catalog://mcp/docker-mcp-catalog/<server-id></code> URIs.
docker mcp profile list	<code>docker mcp profile list</code>	- Lists all local profiles - use <code>docker mcp profile show <id></code> for servers and config in one profile
docker mcp gateway run --transport	<code>docker mcp gateway run --port 8080 --transport streaming</code>	- Serves the gateway over <code>sse</code> or <code>streaming</code> on a TCP port instead of the <code>stdio</code> default, for clients that connect over the network <code>--port</code> connections authenticate with the <code>MCP_GATEWAY_AUTH_TOKEN</code> environment variable
docker mcp catalog create	<code>docker mcp catalog create registry.example.com/mcp/data-tools:latest` `
 --title "Data Analysis Tools" ` --server catalog://mcp/docker-mcp-catalog/sequentialthinking</code>	- Builds a custom catalog from scratch, a profile, or a community registry, choosing exactly which servers it contains - push it with <code>docker mcp catalog push</code> so a team can <code>docker mcp catalog pull</code> the same curated set
docker mcp profile export/import	<code>docker mcp profile export web-dev .docker/mcp-profile.json</code>	- Exports a profile to a JSON file for version-control sharing with teammates - credentials are excluded; recipients configure secrets separately after <code>docker mcp profile import</code> .

References

Official Documentation

1. docker run
2. docker start
3. docker stop
4. docker restart
5. docker pause
6. docker unpause
7. docker rm
8. docker kill
9. docker create
10. docker wait
11. docker rename
12. docker update
13. docker ps
14. docker logs
15. docker exec
16. docker attach
17. docker inspect
18. docker stats
19. docker top
20. docker port
21. docker diff
22. docker cp (container to host)
23. docker build
24. docker images
25. docker pull
26. docker push
27. docker tag
28. docker rmi
29. docker history
30. docker save
31. docker load
32. docker import
33. docker export
34. docker commit
35. docker buildx imagetools inspect
36. docker search
37. Parser directives
38. Multi-stage builds
39. Layer caching
40. .dockerignore
41. Build secrets
42. Combining RUN commands
43. Buildx for multi-platform
44. External cache
45. Inline cache
46. SBOM attestation
47. Provenance attestation
48. Buildx bake
49. docker login
50. docker logout
51. Image digests (immutable)
52. -p (publish)
53. Docker and UFW
54. bridge (default)
55. host
56. none
57. Container network mode
58. overlay
59. macvlan
60. ipvlan
61. nftables firewall backend
62. docker network create
63. docker network ls
64. docker network connect

65. docker network disconnect
67. docker network rm
69. Named volumes
71. tmpfs mounts
73. docker volume create
75. docker volume inspect
77. --memory (-m)
79. --gpus
81. json-file (default)
83. Log rotation (max-size, max-file)
85. journald
87. fluentd
89. gcplogs
91. Dual logging
93. docker inspect
95. docker system df
97. docker version
99. docker debug
101. docker system prune
103. docker container prune
105. docker volume prune
107. image
109. docker compose down
111. docker compose logs
113. docker compose build
115. docker compose restart
117. docker compose start
119. docker compose config
121. docker compose watch
123. env_file
125. Interpolation defaults
127. extends
129. develop (watch)
131. GPU support
133. Scan images
135. Protect the Docker daemon socket
137. User namespace remapping
139. docker dhi catalog list
141. docker dhi attestation list
143. docker dhi customization list
145. docker scout quickview
147. docker scout sbom
149. docker scout attestation add
151. docker scout policy
153. docker context create
155. docker context ls
157. docker context rm
159. daemon.json
161. containerd image store
163. Remote access over TCP with TLS
165. IPv6 for the default bridge network
167. Prometheus metrics endpoint
169. Embedded containerd (experimental)
171. docker swarm join
173. docker service scale
175. docker service ps
177. docker node ls
179. docker stack deploy
181. Build with cloud
183. Set as default builder
185. Build Cloud in CI
66. docker network inspect
68. host.docker.internal
70. Bind mounts
72. Image mounts
74. docker volume ls
76. docker volume rm
78. OOMKilled flag
80. no (default)
82. local
84. syslog
86. gelf
88. awslogs
90. splunk
92. Log tags
94. docker events
96. docker system info
98. Read the daemon logs
100. docker init
102. docker image prune
104. docker network prune
106. docker builder prune
108. docker compose up
110. docker compose ps
112. docker compose exec
114. docker compose pull
116. docker compose stop
118. docker compose run
120. docker compose -f / -p
122. profiles
124. override files
126. COMPOSE_* environment variables
128. include
130. YAML anchors
132. models (Compose AI)
134. Default seccomp profile
136. Rootless mode
138. Content trust (docker trust)
140. docker dhi catalog get
142. docker dhi mirror stop
144. docker scout cves
146. docker scout recommendations
148. docker scout compare
150. docker scout environment
152. docker scout enroll
154. docker context use
156. docker context inspect
158. --context flag
160. Storage drivers (overlay2 and the rest)
162. Live restore
164. Daemon proxy configuration
166. Alternative container runtimes (--runtime)
168. default-stop-timeout (Engine 29.7.0)
170. docker swarm init
172. docker service create
174. docker service update
176. docker service logs
178. docker secret create
180. Connect to cloud builder
182. docker offload start
184. List builders
186. Enable Model Runner

- 187. [docker model pull](#)
- 189. [docker model list](#)
- 191. [docker model status](#)
- 193. [docker model package](#)
- 195. [OpenAI-compatible API](#)
- 197. [Permissions and approval](#)
- 199. [docker mcp profile create](#)
- 201. [--timeout \(-t\)](#)
- 203. [Port range](#)
- 205. [environment](#)
- 207. [docker offload start](#)
- 209. [Docker Documentation — Home](#)
- 211. [Docker Engine Overview](#)
- 213. [docker network Reference](#)
- 215. [docker image Reference](#)
- 217. [Compose `volumes` Reference](#)
- 219. [Compose `configs` Reference](#)
- 221. [Docker Build Cache](#)
- 223. [docker buildx Overview](#)
- 225. [Docker Swarm Overview](#)
- 227. [Swarm Secrets](#)
- 229. [Docker Hub Documentation](#)
- 231. [Docker Security Overview](#)
- 233. [Docker Daemon Configuration \(`daemon.json` \)](#)
- 235. [Docker Remote API / SDK](#)
- 237. [docker scout CLI Reference](#)
- 239. [docker model Reference](#)
- 241. [Docker AI Tools Overview](#)
- 243. [Docker MCP Toolkit — Get Started](#)
- 245. [Docker DHI — Mirror Images](#)
- 247. [Docker Engine 28 Release Notes](#)
- 249. [Logging Drivers](#)
- 251. [Docker Attestations \(SBOM / Provenance\)](#)
- 253. [docker manifest Reference](#)
- 255. [No-new-privileges flag](#)
- 257. [OpenAI Chat Completions API \(Docker Model Runner endpoint\)](#)
- 259. [Docker Offload — About](#)
- 261. [Docker Compose — Using Secrets](#)
- 263. [Docker Engine on Linux — Installation \(Ubuntu\)](#)
- 265. [Docker Desktop for Mac — VirtioFS Performance](#)
- 267. [Docker Desktop — Extensions Marketplace](#)

Specifications & Standards

- 269. [Docker Security Cheat Sheet \(OWASP\)](#)
- 270. [NIST SP 800-190 — Application Container Security Guide](#)

Books

- 271. [Docker: Up & Running, 3rd Edition \(O'Reilly\)](#)
- 272. [Container Security \(O'Reilly — Liz Rice\)](#)

Source Code & Repositories

- 273. [Git commit SHA](#)
- 275. [Distroless :debug variant](#)
- 277. [docker mcp gateway run --transport](#)
- 279. [CVE-2025-62725 — Compose include OCI path traversal](#)
- 281. [BuildKit v0.32.0 Release Notes \(GitHub\)](#)
- 283. [Bret Fisher — Docker Mastery Course Notes](#)
- 285. [Trivy — Vulnerability Scanner for Containers \(GitHub\)](#)
- 287. [Syft — SBOM Generator for Containers \(Anchore\)](#)

- 188. [docker model run](#)
- 190. [docker model rm](#)
- 192. [Inference engines](#)
- 194. [docker model push](#)
- 196. [docker ai \(TUI\)](#)
- 198. [Configure Gordon's tools](#)
- 200. [docker mcp gateway run --transport](#)
- 202. [docker inspect](#)
- 204. [--oom-kill-disable](#)
- 206. [docker-compose v1 removal date](#)
- 208. [Docker MCP Toolkit](#)
- 210. [Docker Compose File Reference](#)
- 212. [Docker Desktop Overview](#)
- 214. [docker volume Reference](#)
- 216. [Compose `networks` Reference](#)
- 218. [Compose `secrets` Reference](#)
- 220. [Compose `include` Reference](#)
- 222. [BuildKit Overview](#)
- 224. [docker buildx bake Reference](#)
- 226. [Swarm Services](#)
- 228. [Docker Registry Overview](#)
- 230. [Docker Hub Automated Builds](#)
- 232. [AppArmor Security Profiles](#)
- 234. [Docker Context Overview](#)
- 236. [Docker Extension SDK](#)
- 238. [Docker Model Runner Overview](#)
- 240. [Docker Model Runner — Supported Models](#)
- 242. [Docker MCP Toolkit Overview](#)
- 244. [Docker DHI — Browse Catalog](#)
- 246. [Docker DHI — Customization](#)
- 248. [Docker Compose Migration \(v1 → v2\)](#)
- 250. [loki Logging Driver \(Grafana\)](#)
- 252. [Docker BuildKit Environment Variables](#)
- 254. [docker trust Reference](#)
- 256. [Docker Compose — Develop \(Watch\) Spec](#)
- 258. [OpenAI Responses API](#)
- 260. [Deprecated and Retired Docker Products and Features](#)
- 262. [Docker Hub Rate Limiting Guide](#)
- 264. [Docker Engine on Linux — Post-install Steps](#)
- 266. [Docker Desktop — Dev Environments \(deprecated\)](#)
- 268. [Sigstore / Cosign — Container Signing](#)

- 274. [Distroless](#)
- 276. [dhctl](#)
- 278. [Docker Compose v2 Release Notes](#)
- 280. [gvisor-tap-vsock \(rootless network driver\)](#)
- 282. [Dive Tool — Exploring Docker Image Layers \(GitHub\)](#)
- 284. [Hadolint — Dockerfile Linter \(GitHub\)](#)
- 286. [Grype — Vulnerability Scanner \(Anchore\)](#)
- 288. [Docker Bench for Security \(GitHub\)](#)

- 289. Dockle — Container Image Linter (GitHub)
- 291. Wolfi Linux — Chainguard's glibc-based undistro
- 293. Docker / Docker CLI (GitHub)
- 295. Docker / Compose (GitHub)
- 297. Docker / Docker Hub Tool (GitHub)
- 299. Docker / Model Runner (GitHub)
- 301. Docker / MCP Gateway (GitHub)
- 303. OCI Image Spec (GitHub)
- 305. Containerd (GitHub)
- 307. awesome-docker (GitHub)
- 309. Dockerfile Best Practices Example Repo (GitHub)

Articles & Tutorials

- 311. Semantic versioning
- 313. Scratch
- 315. Docker Hardened Images (DHI)
- 317. Docker MCP Catalog
- 319. Docker Blog — Official Announcements
- 321. Docker Blog — Gordon GA (Docker AI)
- 323. Docker Blog — Docker Model Runner
- 325. Basil Teo — Docker Compose Evolution: v1 to v5 (2025)
- 327. Ivan Velichko — Container Networking Deep Dive
- 329. Thoughtworks Technology Radar — Docker
- 331. CIS Docker Benchmark
- 333. Podman vs Docker (Red Hat overview)
- 335. Snyk — Docker Security Best Practices (2025)
- 337. Google Cloud — Containers Best Practices
- 339. OpenSSF — Securing the Software Supply Chain (GitHub Actions / Docker)

Video

- 340. Docker — Official YouTube Channel
- 342. Fireship — 100 Seconds of Docker
- 344. Bret Fisher — Docker Compose v2 Deep Dive
- 346. Docker DockerCon 2025 Keynote

- 290. Container Structure Tests (Google)
- 292. nerdctl — containerd CLI with Docker-compatible syntax (GitHub)
- 294. Moby / Moby (Docker Engine — GitHub)
- 296. Docker / Buildx (GitHub)
- 298. Docker / Scout CLI (GitHub)
- 300. Docker / Gordon AI (GitHub)
- 302. Docker / DHI CLI (GitHub)
- 304. OCI Runtime Spec (GitHub)
- 306. Runc — OCI Runtime Reference Implementation (GitHub)
- 308. Docker Samples (GitHub)
- 310. Docker Desktop DHI Demo App (GitHub)

- 312. Alpine Linux
- 314. Chiseled Ubuntu images
- 316. Chiseled Ubuntu images
- 318. Docker Official Images on Docker Hub
- 320. Docker Blog — Hardened Images Go Free (Dec 2025)
- 322. Docker Blog — MCP Toolkit
- 324. Ajeet Raina — Docker MCP CLI Cheat Sheet (2025)
- 326. Earthly Blog — Docker Multistage Build Best Practices (2025)
- 328. Nigel Poulton — Docker Deep Dive (2025 edition)
- 330. Slim.AI — Container Minification
- 332. Chainguard Images (hardened minimal images)
- 334. The Docker Book (James Turnbull)
- 336. Red Hat — Container Security Best Practices
- 338. 12-Factor App Methodology

- 341. TechWorld with Nana — Docker Tutorial for Beginners (2M+ views)
- 343. NetworkChuck — Docker Networking (600K+ views)
- 345. KodeKloud — Docker for Absolute Beginners